

Аудит системы дебатов ai-os-new

Комплексный анализ архитектуры, логики и качества кода подсистемы дебатов. Обнаружено более 100 проблем, включая критические race conditions, несовместимые типы данных и неработающие метрики анализа.

Проект: n95887174-source/ai-os-new
Дата: 22 июня 2026
Область: Debates, Runtime, UI, State

100+
ПРОБЛЕМ НАЙДЕНО

10
КРИТИЧЕСКИХ

26
СЕРЬЁЗНЫХ

1. Резюме аудита	3
1.1 Статистика находок	3
2. Корневые причины: почему "всё работает ужасно"	3
2.1 Два параллельных пути с несовместимыми типами	3
2.2 Оркестратор -- пустышка	4
2.3 Английские эвристики для русских дебатов	4
2.4 Race conditions	4
2.5 UI не реактивен	4
3. Критические проблемы	5
3.1 Типы Claim несовместимы	5
3.2 Топологическая сортировка удаляет участников	5
3.3 Abort Controllers перезаписываются	5
3.4 Branching merge с конфликтами	5
3.5 Budget race condition	6
3.6 Три несовместимых DebateSessionState	6
3.7 Voting через globalThis	6
3.8 Infinite render loop	6
3.9 Strategy type casting	6
3.10 Orchestrator -- no-op	6
4. Серьёзные проблемы (HIGH)	7
4.1 Ядро рантайма	7
4.2 Топология и оценка	7
4.3 UI-компоненты	8
4.4 Сервисный слой	8
5. Архитектурные и средние проблемы	9
5.1 Детекция противоречий	9
5.2 Governor	9
5.3 Промпты и LLM	9
5.4 Runtime Adapter	10

5.5 State Management	10
5.6 Room и Session	10
6. Мелкие проблемы (LOW)	11
7. Рекомендации и план действий	12
7.1 Фаза 1: Критические исправления (неделя 1-2)	12
7.2 Фаза 2: Архитектурные исправления (неделя 2-4)	12
7.3 Фаза 3: Качество и i18n (неделя 4-6)	13
7.4 Фаза 4: Полировка (неделя 6+)	13

1. Резюме аудита

Проведён глубокий аудит всей подсистемы дебатов в проекте ai-os-new. Проанализировано более 30 ключевых файлов: ядро рантайма (engine, orchestrator, room, session), сервисный слой (debate-service, llm-caller, prompt-builder, runtime-adapter), governor (claim-graph, contradiction-detector, claim-extractor), инфраструктура (topology, branching, budget, evaluator, consensus), UI-компоненты (DebatePanel, DebateLive, DebateArena и другие), state management (debate-state, debate-runtime-state, debateLiveStore) и контрактные типы.

Обнаружено более 100 проблем различных категорий и уровней серьёзности. Основная причина, по которой система "всё работает, но как-то ужасно", заключается не в отдельных багах, а в системных архитектурных проблемах, которые накапливают эффект друг на друга. Ниже приведён детальный разбор по каждой категории.

1.1 Статистика находок

Уровень	Кол-во	Описание
CRITICAL	10	Краш, потеря данных, поломка ядра
HIGH	26	Логические ошибки, race conditions
MEDIUM	37	Некорректное поведение, проблемы качества
LOW	24	Мелкие проблемы, улучшения

Таблица 1. Распределение находок по уровню серьёзности

2. Корневые причины: почему "всё работает ужасно"

Формально код не падает с исключением на каждом шагу, и дебаты действительно происходят от начала до конца. Однако совокупность архитектурных проблем создаёт эффект "работает, но ужасно". Пять корневых причин объясняют этот феномен.

2.1 Два параллельных пути с несовместимыми типами

Система содержит два параллельных пути выполнения: legacy-сервис (debate-service.ts, ~980 строк) и новый рантайм (debate-runtime/). Они связаны через DebateRuntimeAdapter, но используют фундаментально несовместимые типы данных. DebateSessionState определён в трёх разных файлах с разными формами. Governor Claim содержит поля speaker/role/status, а Runtime Claim содержит agentId/confidence/evidence. Когда данные пересекаются между модулями, поля оказываются undefined, арифметика выдаёт NaN, а метрики silently corrupt. Это корневая причина большинства

"странностей" в поведении аналитики и консенсуса. Каждая новая фича, затрагивающая оба пути, удваивает вероятность багов.

2.2 Оркестратор -- пустышка

IDebateOrchestrator объявляет 8 типов событий (agent:thinking, agent:responded, consensus:reached и т.д.), но реальный оркестратор (DebateOrchestrator, всего 45 строк) выдаёт только round:start и round:end. Вся реальная работа -- вызовы LLM, обработка ответов, оценка консенсуса, управление бюджетом -- выполняется в DebateEngine.startSession(), методе на 800+ строк. Оркестратор -- это пустой слой абстракции, создающий ложное впечатление модульности. Чтобы понять, что происходит в дебате, нужно читать всю цепочку: engine -> room -> session -> context -> orchestrator, при этом 90% логики сосредоточено в engine.

2.3 Английские эвристики для русских дебатов

Все промпты принудительно заканчиваются "Respond in Russian", но вся аналитическая инфраструктура заточена под английский язык. Socratic Quality Gate проверяет тривиальные вопросы по английским паттернам ("can you elaborate", "what do you mean") -- русские аналоги никогда не детектируются, и Сократ может задавать "Можете уточнить?" без штрафа. Метрики качества (evidence detection ищет "according to", "study shows"; constraint compliance ищет "I think", "maybe"; speculation detection ищет "perhaps", "likely") -- русские дебаты получают нулевые или случайные оценки. Синоним-группы в duplicate detection включают неполный набор русских слов. Регулярные выражения для очистки текста в claim-extractor не включают букву "ё". В результате система анализирует русские дебаты как английские, и метрики качества, оригинальности и сходимости по сути случайны.

2.4 Race conditions

Движок использует общие Map-объекты (llmFailureCount, sessionAbortControllers, participantKeyMap) без какой-либо синхронизации. llmFailureCount индексируется по agentId без sessionId, что вызывает перекрёстное загрязнение между сессиями: сбой агента "alice" в сессии А снижает количество ретраев для "alice" в сессии В. sessionAbortControllers хранит один контроллер на sessionId, перезаписывая его для каждого агента в раунде. Отмена сессии отменяет только последнего активного агента, остальные продолжают потреблять токены. Budget.canProceed() и recordUsage() имеют классическую TOCTOU race condition: два агента могут одновременно пройти проверку и превысить бюджет. Эти race conditions непредсказуемы и зависят от тайминга, что объясняет "иногда работает нормально, иногда ужасно".

2.5 UI не реактивен

DebateLivePanel вызывает engine.getAllSessions() в теле рендера (не в хуке), создавая новый массив при каждом рендере и потенциально вызывая бесконечный цикл. DebateBranchPanel использует polling каждые 5 секунд вместо подписок на события. DebateSidebar загружает комнаты только при монтировании и не обновляется. Состояние DebateSession возвращается из сервиса без

клонирования, что нарушает контракт иммутабельности React. Zustand store создаёт new Map() на каждое streaming-событие, вызывая excessive re-renders в CircularLayout и SpeakerNode. В совокупности это создаёт ощущения "лагающего" и "неотзывчивого" интерфейса, даже когда backend работает корректно.

3. Критические проблемы

Критические проблемы приводят к крашу, потере данных или фундаментально неверному поведению. Каждая требует немедленного исправления.

3.1 Типы Claim несовместимы

[CRITICAL] Cross-module Claim type incompatibility

debate-governor/types.ts vs debate-runtime.ts

Governor Claim: {speaker, role, status, supportCount}. Runtime Claim: {agentId, confidence, evidence}.

Evaluator.scoreArguments() читает c.agentId и c.confidence из governor-claims -> undefined -> NaN во всех арифметических выражениях. Оценки и консенсус полностью сломаны при пересечении модулей.

3.2 Топологическая сортировка удаляет участников

[CRITICAL] Silent node dropping on cycle

debate-topology.ts:97-126

При наличии цикла topologicalSort() логирует предупреждение, но возвращает усечённый результат. Участники в цикле навсегда исключены из всех раундов. Нет ошибки, нет повтора, нет уведомления пользователя -- дебат продолжается с недостающими голосами.

3.3 Abort Controllers перезаписываются

[CRITICAL] Single AbortController per session overwritten per agent

debate-engine.ts:81,358

sessionAbortControllers хранит один AbortController на sessionId. В цикле раунда каждый агент перезаписывает контроллер. Отмена отменяет только последнего агента, остальные LLM-вызовы продолжают потреблять токены до таймаута (30с). Pause вообще не прерывает LLM-вызовы.

3.4 Branching merge с конфликтами

[CRITICAL] Merge includes conflicting arguments despite detection

debate-branching.ts:61-82

Метод merge обнаруживает конфликты (одинаковые agentId+round на обеих ветках), но unconditionally объединяет ВСЕ аргументы, включая конфликтующие. Caller видит success: true при наличии неразрешённых конфликтов с повреждёнными данными в результате.

3.5 Budget race condition

[CRITICAL] TOCTOU race in canProceed/recordUsage

debate-budget.ts:44-66

canProceed() читает `_tokensUsed`, `recordUsage()` затем мутирует его. Между проверкой и записью другой параллельный вызов тоже проходит проверку. Оба записывают, превышая бюджет. Нет мьютекса или атомарной операции.

3.6 Три несовместимых DebateSessionState

[CRITICAL] Triple incompatible type definitions

debate-state.ts / debate-runtime-state.ts / debate-runtime.ts

Тип определён в трёх файлах с разными формами. Компоненты импортируют любой вариант -- несоответствие вызывает runtime crashes (missing fields, undefined access) при перестройке.

3.7 Voting через globalThis

[CRITICAL] Voting uses globalThis instead of service

DebateTabContent.tsx:222-224

Кнопка голосования использует `(globalThis as any).__debateService?.recordHumanVote(...)`. Обходит сервисный слой, не типобезопасно, silently no-op если свойство не существует. `Prop setHumanVotes` не подключён -- состояние голосования родителя не обновляется.

3.8 Infinite render loop

[CRITICAL] getAllSessions() called on every render

DebateLivePanel.tsx:23-32

`engine.getAllSessions()` в теле компонента создаёт новый массив при каждом рендере. `useEffect` с зависимостью от `sessions` запускает рендер заново -- потенциальный бесконечный цикл.

3.9 Strategy type casting

[CRITICAL] Strategy cast omits jury_trial and cross_examination

DebatePanel.tsx:525-526,802-803

`onStrategyCallback` кастит значение к union type, пропуская `jury_trial` и `cross_examination`. Выбор "Jury Trial" в UI молча искажается на уровне типа -- значение теряется.

3.10 Orchestrator -- no-op

[CRITICAL] Orchestrator yields only round:start/end of 8 event types

debate-orchestrator.ts:20-40

Все agent-level события -- мёртвый код в типовой системе. Оркестратор не ссылается на LLM, session или agents -- это round-counter, а не оркестратор. Вся работа в `engine.startSession()`.

4. Серьёзные проблемы (HIGH)

4.1 Ядро рантайма

[HIGH] llmFailureCount cross-session pollution

debate-engine.ts:79,354,504

Индексируется по agentId без sessionId. Успех агента в сессии В сбрасывает счётчик ошибок в сессии А.

[HIGH] getRankedProviders status check always false

debate-engine.ts:394

k.status === "active" всегда false -- поле status не существует в возвращаемом типе. Весь ranked-providers fallback мёртв.

[HIGH] resumeSession swallows errors, emits event prematurely

debate-engine.ts:600-612

SESSION_RESUMED излучается ДО фактического resume. Ошибка глотаётся через .catch(). UI показывает "resumed" но сессия может сразу упасть.

[HIGH] pauseSession does not abort in-flight LLM calls

debate-engine.ts:590-598

Пауза останавливает раунд, но текущий LLM-вызов продолжает выполняться до таймаута, потребляя токены. Только cancelSession вызывает abort().

[HIGH] API keys stored in plaintext memory

debate-engine.ts:78,387

participantKeyMap хранит сырые ключи. При memory dump или debug log -- exposed.

[HIGH] Multi-agent history collapses to 2-party format

debate-engine.ts:417-419

BCE non-self сообщения получают роль "user". В 4-агентном дебате агент С видит аргументы А, В, D как от одного пользователя. LLM не различает оппонентов -- качество ответов деградирует.

4.2 Топология и оценка

[HIGH] red-blue topology max 3 rounds

debate-topology.ts:73-81

attackers -> defenders -> judges. Нет цикла. 10-раундовый red-blue дебат невозможен.

[HIGH] Double-counting in scoring formula

debate-evaluator.ts:25-35

persuasiveness включает avgConfidence + coherence. overall включает persuasiveness + прямые веса. Эффективные веса unknowable, одно поле может доминировать.

[HIGH] factuality measures confidence, not factuality

debate-evaluator.ts:26

factuality = min(1, avgConfidence + 0.1). Уверенная ложь = 0.95+. Нет проверки фактов.

4.3 UI-компоненты

[HIGH] 400 lines duplicated mobile/desktop JSX

DebatePanel.tsx:412-962

DebateTabContent извлечён, но НЕ используется. Багфикс в одном пути не применяется к другому.

[HIGH] Scroll effect triggers on every render

DebateChat.tsx:20-24

args prop создаёт новый массив при каждом parent render -> scroll на каждом рендере.

[HIGH] eslint-disable hides missing deps in useEffect

DebatePanel.tsx:157-198

Ссылается на t, prevRoundRef, scrollRef и другие vars не в deps. Stale closure risk.

[HIGH] 5-second polling instead of event subscriptions

DebateBranchPanel.tsx:26-29

До 5с задержки UI при изменении веток.

[HIGH] Hardcoded Russian strings break i18n

DebateVerdictPanel/DebateBranchPanel

"Вердикт дебатов", "Тип", "Баланс" вместо t(). Сломанный i18n.

4.4 Сервисный слой

[HIGH] Unsafe double type assertion for verdict

debate-service.ts:667

activeSession as unknown as DebateSessionSnapshot -- типы имеют разные формы. ConclusionEngine получает некорректные данные.

[HIGH] Mutates input participants array

debate-service.ts:269

participants.forEach(p => p.constraint = ...) мутирует входной массив. При повторном использовании participants ограничения остаются от предыдущего дебата.

[HIGH] calculateConfidence is trivial word-count heuristic

debate-stop-conditions.ts:5-16

+0.2 за 50-300 слов, -0.2 за >500 слов. 500+ слов data-rich ответ = 0.4. 100 слов пустой ответ = 0.7. Используется как мера качества аргумента повсюду.

5. Архитектурные и средние проблемы

5.1 Детекция противоречий

[MEDIUM] Massive false positives in isContradictory()

debate-consensus.ts:143-184

Антоним-пары: "temperature is high" vs "quality is low" = contradictory. Негация: "I do not like coffee" vs "sky is blue" = contradictory (есть "not"). Числа: "Revenue grew 2.1%" vs "Revenue grew 2.2%" = contradictory.

[MEDIUM] contradictionDensity counts resolved conflicts

debate-consensus.ts:22

После разрешения всех противоречий density > 0. Вводит в заблуждение downstream consumers.

[MEDIUM] O(n*n*m*m) complexity from Array.includes

contradiction-detector.ts:47

Использовать Set.has() для O(n) lookup. Текущая сложность экспоненциальна для длинных дебатов.

5.2 Governor

[MEDIUM] Module-level mutable ID counters

contradiction-detector.ts:4, claim-extractor.ts:3

_contradictionCounter и _claimCounter персистируют между сессиями в Next.js SSR/HMR.

Параллельные дебаты делят один счётчик -> colliding IDs.

[MEDIUM] Regex missing letter yo (ё)

claim-extractor.ts:30

/[^a-za-я0-9\s]/g не включает ё. Слова с ё ломают дедупликацию.

[MEDIUM] No duplicate edge or self-loop prevention

claim-graph.ts:15-23

Повторные вызовы addEdge() создают дубликаты, загрязняя обходы графа.

5.3 Промпты и LLM

[MEDIUM] Hardcoded "Respond in Russian" in all prompts

debate-prompt-builder.ts:105,183,192,200

Независимо от языка топики. Английские дебаты принудительно на русском. Смешивание языков в промптах деградирует качество LLM-ответов.

[MEDIUM] Non-deterministic parent selection in argument tree

debate-prompt-builder.ts:137

Math.random() -- при одинаковых данных каждый раз разный parent. Нерепродуцируемость.

[MEDIUM] Undocumented temperature/model offset from agent ID hash

debate-llm-caller.ts:127,162-163

Температура модифицируется на ± 0.18 на основе хеша ID. Не документировано, не контролируемо.

[MEDIUM] Consensus uses "most recent" not "highest confidence" args

debate-consensus-generator.ts:10-16

Комментарий говорит "highest confidence", код делает sort by timestamp DESC + slice(-4).

5.4 Runtime Adapter

[MEDIUM] Fire-and-forget startSession with error handling gap

debate-runtime-adapter.ts:153-159

Ошибка до возврата Promise не перехватывается. finalize() вызывается и при успехе и при ошибке, но при ошибке статус "active" -> saveToHistory() пропустит сессию.

[MEDIUM] Unsafe type assertion for exportLegacySession

debate-runtime-adapter.ts:69-71

engine as { exportLegacySession?: ... } для доступа к методу, не в IDebateEngine.

5.5 State Management

[MEDIUM] New Map() on every streaming event causes excessive re-renders

debateLiveStore.ts:46-66

Каждый event handler создаёт new Map() -> Zustand видит новую ссылку -> все selectors re-render.

[MEDIUM] getHistory() returns internal array without cloning

debate-service.ts:821-823

Consumer может мутировать внутренний массив сервиса.

[MEDIUM] No destroy() method -- memory leak per session

debate-session-context.ts:9-18

Создаёт 4 объекта (ConsensusEngine, Timeline, Orchestrator, ConclusionEngine), но никогда их не уничтожает.

5.6 Room и Session

[MEDIUM] injectEvent creates dual records

debate-room.ts:161-187

Push в injectedEvents И applyOverride -> push в overrides. Двойная запись с разными структурами.

[MEDIUM] step() runs full resume, not single step

debate-room.ts:109-119

Назван "step", но вызывает resume() -- запускает дебат до конца. Нет single-step.

[MEDIUM] paused -> deliberating skips active phase

debate-session.ts:21

VALID_TRANSITIONS.paused включает deliberating. Может пропустить critical initialization.

[MEDIUM] Same multi-agent history collapse in legacy service

debate-service.ts:363-372

buildHistoryMessages() -- все non-self сообщения = "user". Контекст LLM не multi-party.

6. Мелкие проблемы (LOW)

Ниже приведена сводная таблица проблем низкого уровня серьёзности. Каждая ухудшает качество кодовой базы или пользовательский опыт, но не является критичной.

Файл	Проблема
debate-engine.ts:24	Дублирование "maybe" в hedgingMarkers regex
debate-engine.ts:667	saveSnapshot() перезаписывает createdAt на Date.now()
debate-engine.ts:356	Off-by-one: MAX_RETRIES+1 попыток вместо MAX_RETRIES
debate-orchestrator.ts:42	destroy() очищает aborted всех сессий
debate-room.ts:101-107	stop() вызывает двойной SESSION_CANCELLED event
debate-room.ts:97	resume() awaits void -- вводящая в заблуждение async сигнатура
debate-topology.ts:55-71	tree-of-thought BFS молча пропускает cycled nodes
debate-branching.ts:96-97	Rollback cost = confidence * 0.001 -- бессмысленно
debate-branching.ts:74	Merge создаёт shared mutable references
debate-branching.ts:25	branches Map растёт бесконечно
debate-consensus.ts:35-38	FIFO вместо LRU eviction в confidence graph
debate-consensus.ts:62-73	Embedding cache keyed on full text -- rapid churn
debate-evaluator.ts:10-18	Rebuttal detection: "butter is delicious" = rebuttal
debate-evaluator.ts:40	argumentQuality = count * 0.1: 10 плохих args = 1.0
claim-graph.ts:57 vs debate-types.ts:261	Две несовместимые jaccardSimilarity
debate-metrics.ts:202	Evidence detection -- только английские паттерны
debate-duplicate-detection.ts:3-21	Неполные synonym groups, в основном английские
DebateSessionStrategy type	cross-examination и cross_examination оба валидны
CircularLayout.tsx:13	Hardcoded RADIUS=200 -- не адаптивен
TournamentPanel.tsx:53-68	Участник может дебатировать сам с собой

debate-store.ts:12-17	JSON-поля без schema validation
HistoricalFiguresPicker.tsx	Her focus trap -- accessibility
DebateSessionStrategy	redundant cross_examination / cross-examination

Таблица 2. Сводка проблем низкого уровня

7. Рекомендации и план действий

На основании проведённого аудита рекомендуется следующий приоритизированный план. Порядок определяется не только серьёзностью, но и взаимозависимостью: исправление корневых архитектурных проблем часто устраняет множество производных багов.

7.1 Фаза 1: Критические исправления (неделя 1-2)

Первый приоритет -- унификация типов и устранение краш-багов. Без этого любые другие исправления могут вводить новые проблемы из-за несовместимости типов.

- Создать единый каноничный тип Claim, разделяемый между Governor и Runtime. Добавить adapter/mapper с runtime validation при пересечении модулей.
- Переключить sessionAbortControllers на Map<sessionId, Map<agentId, AbortController>>. На cancel/abort -- вызывать abort() на BCEX контроллерах сессии.
- Добавить destroy() в DebateSessionContext для очистки всех sub-resources (consensus engine, orchestrator, conclusion engine).
- Исправить DebateTabContent voting: заменить globalThis.__debateService на прямое использование debateService. Подключить setHumanVotes prop.
- Исправить DebateLivePanel: перенести getAllSessions() в useMemo или useEffect, устранить render loop.

7.2 Фаза 2: Архитектурные исправления (неделя 2-4)

Второй приоритет -- устранение двойного пути и race conditions. Требуется более глубокой переработки, но существенно повысит надёжность и поддерживаемость.

- Индексировать llmFailureCount по sessionId:agentId вместо просто agentId.
- Внедрить атомарную budget reservation: replace canProceed()/recordUsage() на reserveAndRecord() с compare-and-swap паттерном.
- Переписать debate-history messages: использовать multi-party format (agent name в content) или switch на multi-agent chat API если поддерживается провайдером.
- Принять решение: выбрать ОДИН execution path (legacy ИЛИ runtime) и deprecate другой. Dual-path maintenance cost слишком высока для solo-разработчика.

- Либо сделать Orchestrator реально управляющим (переместить LLM logic из engine), либо удалить интерфейс и встроить ground logic напрямую в engine.

7.3 Фаза 3: Качество и i18n (неделя 4-6)

Третий приоритет -- сделать метрики работающими для русских дебатов и исправить UI.

- Удалить хардкод "Respond in Russian" из всех промптов. Добавить параметр language в DebateConfig.
- Переписать Socratic Quality Gate: добавить русские паттерны тривиальных вопросов.
- Расширить evidence detection, constraint compliance и speculation detection на русский язык.
- Заменить keyword-based contradiction detection на embedding-distance или LLM-assisted. Текущая false positive rate неприемлема.
- Исправить regex cleanup: включить ё во все регулярные выражения.
- Удалить 400 строк дублированного JSX, использовать DebateTabContent.
- Заменить polling в DebateBranchPanel на event-driven подписки.
- Заменить хардкод русских строк в VerdictPanel и BranchPanel на t().

7.4 Фаза 4: Полировка (неделя 6+)

Заключительная фаза -- производительность и предотвращение регрессий.

- Заменить Array.includes на Set.has в contradiction-detector для $O(n)$ вместо $O(n*m)$.
- Добавить self-loop и duplicate-edge prevention в claim-graph.addEdge().
- Внедрить LRU eviction вместо FIFO в confidence graph.
- Добавить schema validation при десериализации DebateSessionRecord.
- Клонировать массивы в getHistory(), getArguments() и DebateMemory.getAllSteps().
- Добавить error boundaries вокруг DebateChat, DebateAnalytics, CollabDebatePanel.
- Убрать eslint-disable и корректно указать все зависимости в useEffect.
- Сделать CircularLayout адаптивным вместо hardcoded RADIUS=200.